

Final Project Report

Car Racing Game



AICT LAB REPORT:

Semester project

Submitted to:

SIR AFEEF OBAID

Submitted by:

MUHAMMAD TAHA KHALID (2024-CE-20)

MUHAMMAD AFNAN ABBAS (2024-CE-03)

UMER HAYAT (2024-CE-48)

HAIDER ARSHAD (2024-CE-36)

Table of Contents:

- 1. Abstract**
- 2. Introduction**
- 3. Implementation**
- 4. Challenges**
- 5. Output Results**
- 6. Conclusion**
- 7. References**

Abstract:

This report describes the creation of a 2D car racing game developed in Snap! by Berkeley. The game is designed to be engaging and fun, starting with a menu screen where players can customize their experience by choosing car skins, adjusting settings like volume and difficulty, and starting the game. The gameplay involves dodging incoming cars, collecting powerups, and avoiding collisions. Players can use a jumping ability to navigate obstacles, and the scoring system adapts based on the chosen difficulty. Key features include animations, sound effects, and dynamic gameplay mechanics. Challenges such as debugging and integrating powerups were successfully addressed, resulting in a polished game.

Introduction:

The car racing game is a simple yet entertaining 2D game built using Snap!, a visual programming language developed by Berkeley. The game offers an engaging top-down view where players control a stationary car, dodging incoming vehicles and

collecting powerups. This project was developed to provide a fun and customizable gaming experience for players of all ages.

The main features of the game are:

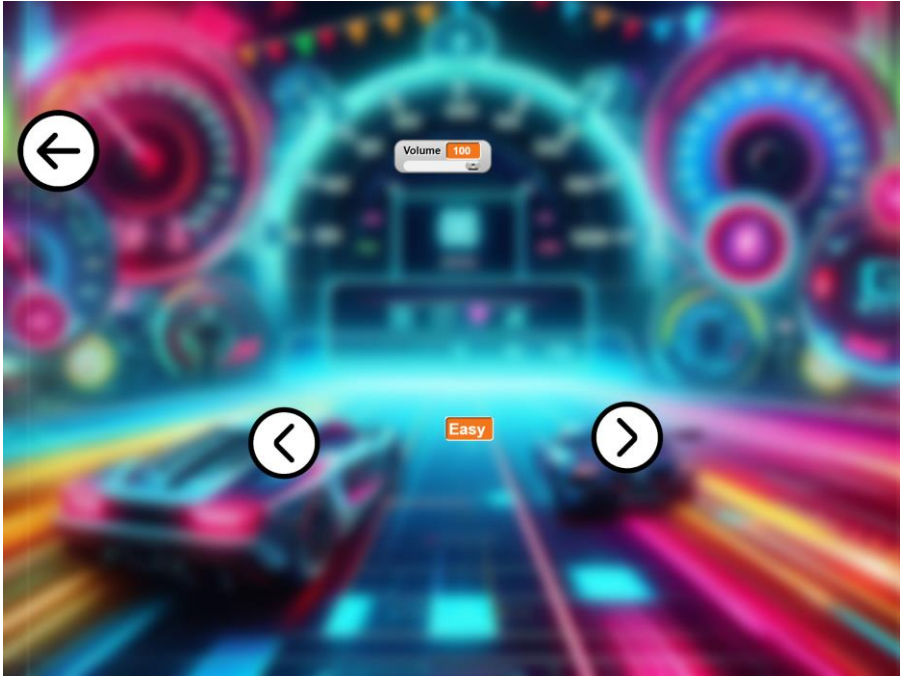
- Menu Screen: A user-friendly interface with three buttons:



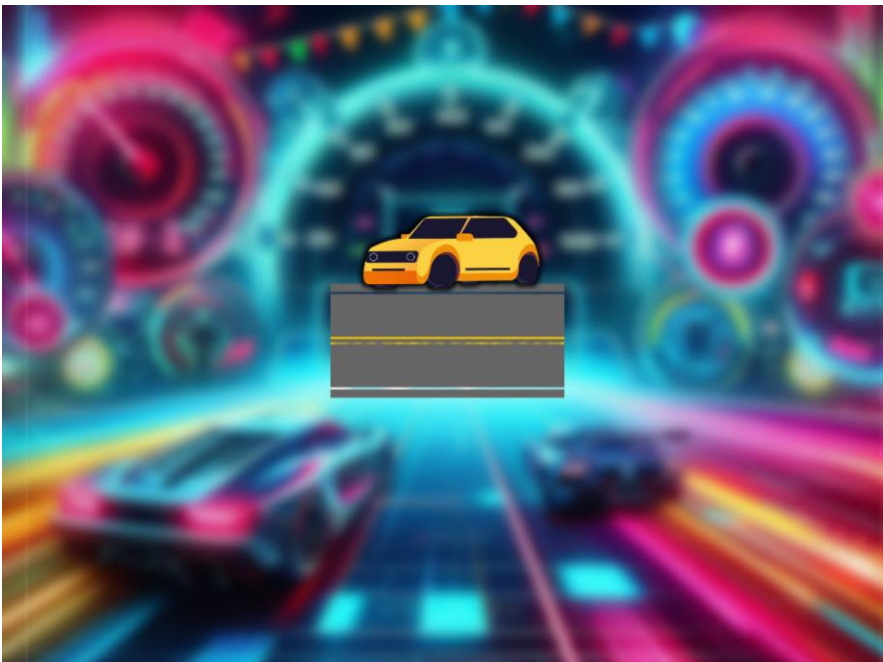
- Cars: Players can select from different car designs or skins to personalize their gameplay.



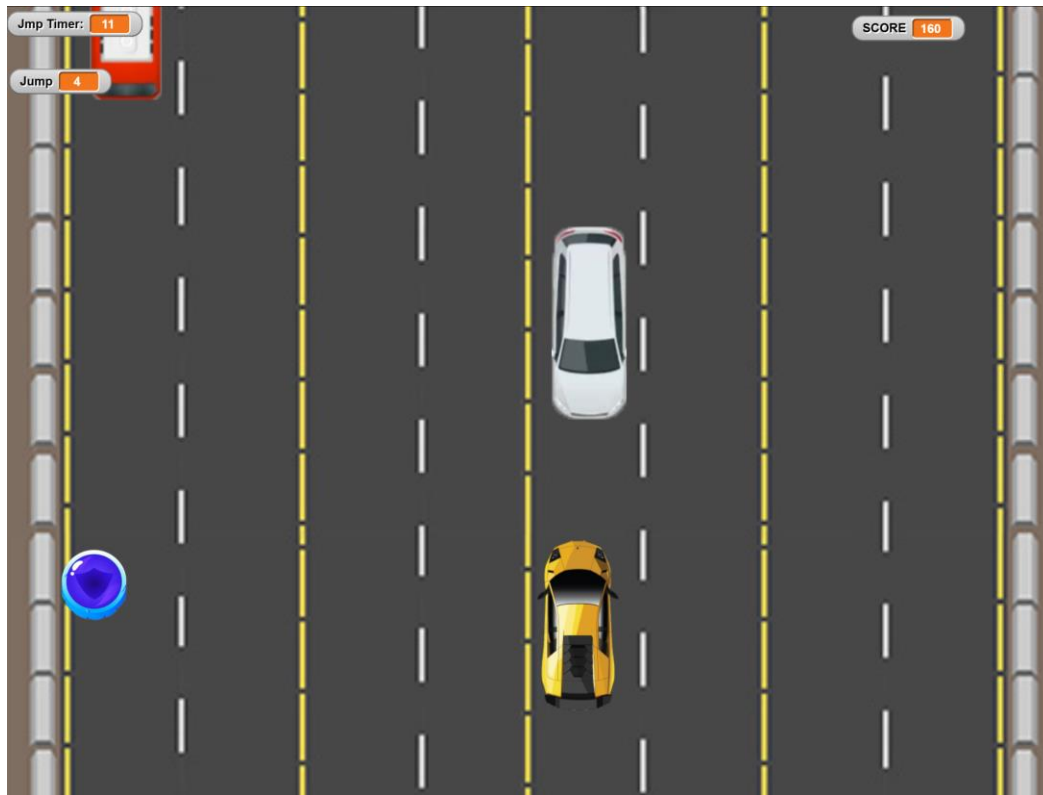
- - Settings: Options to adjust volume and difficulty levels.



- - Start: Begins the game with a loading animation.



- - Gameplay Mechanics: Players dodge incoming cars and use jumps to avoid collisions while collecting powerups to enhance their performance.



- - Scoring System: Scores increase dynamically based on the difficulty level.
- - Sound System: Background music and sound effects add to the immersive experience.

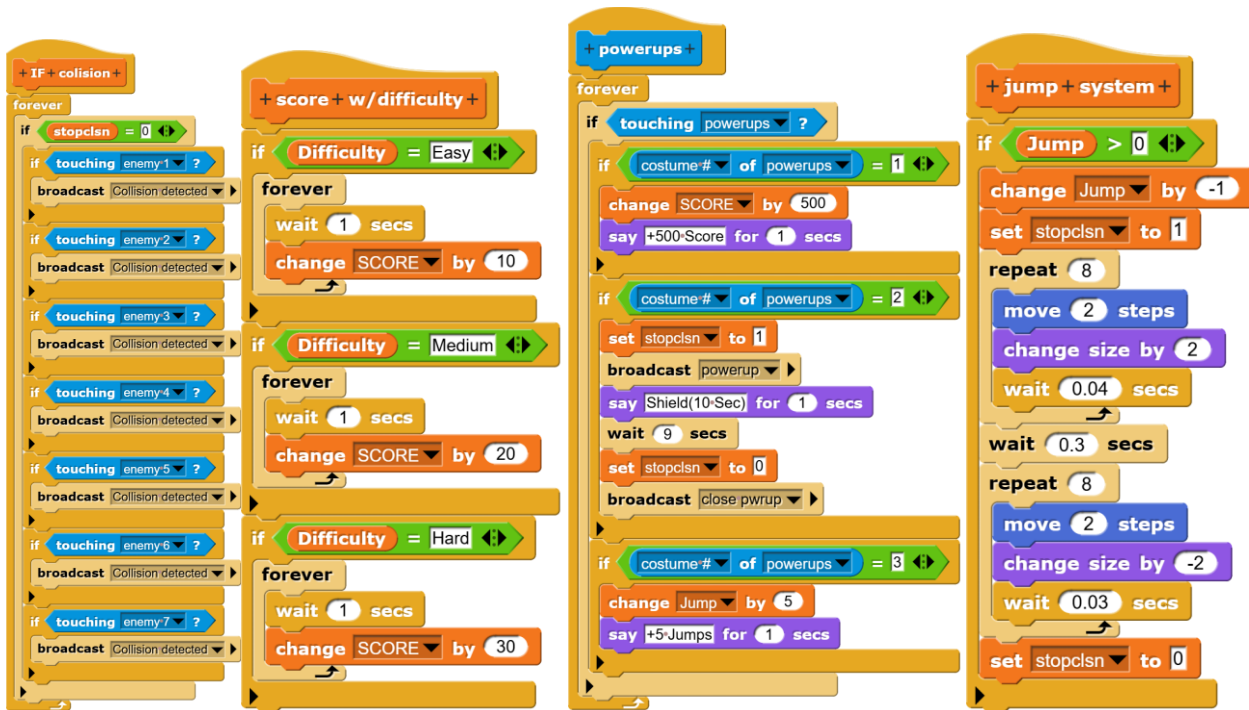
Implementation:

Development Platform:

The game was developed using Snap!, a visual, block-based programming language ideal for creating interactive games. Snap! provides an intuitive environment for building games with its drag-and-drop functionality, making it suitable for implementing animations, event-driven programming, and dynamic gameplay.

Custom Blocks:

Creating custom blocks in Snap! (Berkeley) can significantly enhance the development of a car racing game, making the programming process more efficient and organized. Custom blocks allow you to encapsulate frequently used code snippets into single commands, promoting modularity and reusability. For instance, you can create custom blocks to handle different aspects of the game, such as "Accelerate," "Decelerate," "Steer Left," and "Steer Right," each encapsulating the specific logic for controlling the car. Additionally, custom blocks can manage collision detection, lap counting, and power-up mechanics, ensuring that the game's core functionalities are neatly separated and easy to modify or expand. By using custom blocks, you can streamline the game development process, improve code readability, and facilitate troubleshooting, ultimately creating a more dynamic and engaging car racing game.



Menu Design:

The game starts with a menu screen that contains three buttons:

1. **Cars:** Allows players to choose from different car designs or skins to personalize their gameplay.
2. **Settings:** Players can adjust the game's volume and select the difficulty level (e.g., Easy, Medium, Hard).
3. **Start:** Begins the game, accompanied by a brief loading animation that builds anticipation.

Core Gameplay Mechanics

- - **Stationary Car Control:** The player's car remains in the center of the screen while cars approach from the opposite side.
- - **Dodging:** Players must move the car left or right to avoid collisions.
- - **Jumping Mechanic:** Players can jump over incoming cars. Each jump has a cooldown of 15 seconds to ensure balanced gameplay.

Powerups:

Powerups appear periodically during gameplay and provide specific advantages:

1. **Jump Powerup:** Adds +5 jumps to the player's jump count.
2. **Score Powerup:** Instantly increases the player's score by +500 points.
3. **Shield Powerup:** Grants the player a shield for 10 seconds, allowing them to avoid collisions.

Scoring System:

The scoring system is tied to the difficulty level:

- Higher difficulty levels increase the score at a faster rate.
- Players are rewarded for their skill and endurance, encouraging them to take on greater challenges.

Collision Handling:

- - If the player collides with an incoming car, the car explodes, and a "Try Again" button appears on the screen.

- - The explosion animation and sound effects create a dramatic impact, motivating players to retry.

Sound System:

The game includes:

- **Background music** that plays during gameplay.
- **Sound effects** for key actions such as collecting powerups, jumping, and colliding.

Challenges:

1. Animations:

- Developing smooth animations for loading screens, car movements, and collisions was challenging in Snap! due to its visual programming constraints.

2. Bug Fixes:

- Resolving issues with unexpected collisions and powerup behaviors required careful debugging and testing.

3. Collision Handling:

- Ensuring accurate collision detection for incoming cars and powerups involved fine-tuning the game logic.

4. Border Control:

- Preventing cars and powerups from moving outside the game boundaries was critical to maintaining gameplay consistency.

5. Powerup Integration:

- Designing a balanced system for powerup spawning and timing added depth to the gameplay but required significant iteration.

6. Custom Blocks:

- Snap! allowed for the creation of custom blocks to handle repetitive tasks such as spawning cars, detecting collisions, and managing powerups efficiently.

Output Results:

- 1:** The game launches with a visually appealing menu screen that allows players to customize their experience.
- 2:** Smooth transitions and animations enhance the overall gameplay experience.
- 3:** Players can dodge cars, collect powerups, and jump effectively to avoid collisions.
- 4:** The scoring system dynamically adjusts based on the difficulty level, rewarding players appropriately.
- 5:** The sound system adds an immersive layer to the game, with background music and effects for key actions.
- 6:** On collision, a dramatic explosion animation and sound effect occur, followed by a "Try Again" option.

Conclusion:

The car racing game developed in Snap! successfully combines intuitive gameplay, engaging mechanics, and a customizable user experience. The challenges faced during development were effectively addressed, resulting in a polished and enjoyable game. By leveraging Snap!'s visual programming capabilities, the game demonstrates the potential of block-based programming for creating dynamic and interactive applications.

Future Improvements:

- - Adding **more car designs** and tracks to increase variety.
- - Introducing **multiplayer modes** for competitive gameplay.

- - **Expanding powerup** options to include new abilities like speed boosts or temporary invincibility.
- - **Refining animations** and enhancing visual effects further.

References:

1. **Snap! by Berkeley** - <https://snap.berkeley.edu> (Coding)
2. <https://pixabay.com/music/search/racing/> (Sound Effects)
3. <https://www.shutterstock.com/video/search/2d-car-top> (GUI)